AI PORTFOLIO PROJECT

Personal Assistant RAG

RAG

Document Intelligence Engine

Ingest any local document. Ask anything. Get grounded answers with source references.

Streamlit · LangChain · OpenAI GPT-4o-mini · FAISS · HuggingFace Embeddings · Python

EXECUTIVE SUMMARY

Personal & enterprise documents remain **locked, unsearchable, and siloed.** This project unlocks them.

5+

File Types Supported
(PDF, DOCX, XLSX, CSV, TXT)

FAISS

Local Vector Index
for Fast Semantic Search

100%

Grounded Answers
With Source References

Ask anything about your documents. Get answers grounded in your data — never hallucinated.

THE PROBLEM & SOLUTION

THE PROBLEM

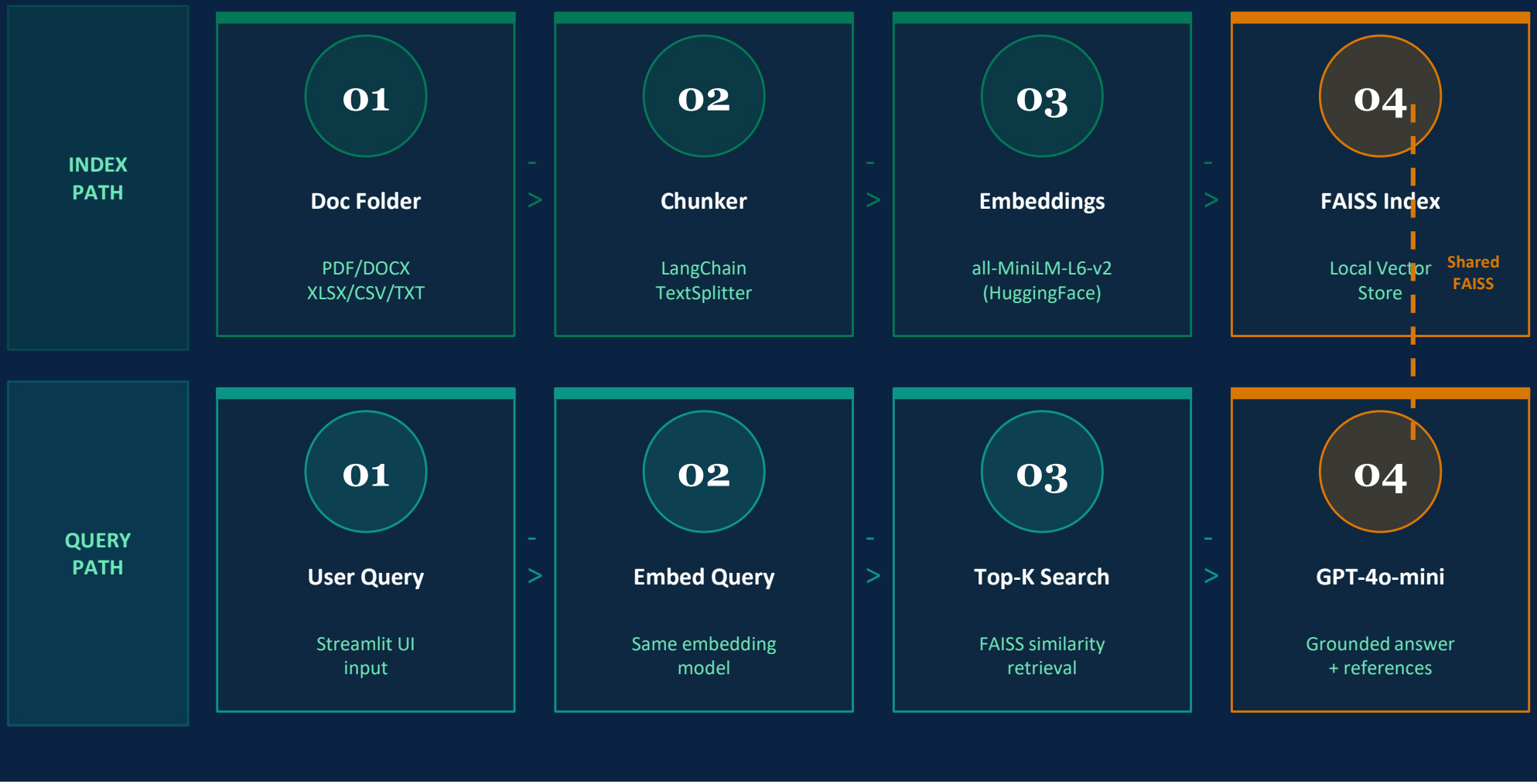
1. Documents scattered across PDF, Word, Excel, CSV, TXT files
2. Manual search is slow and misses semantic context
3. No unified way to query across file types
4. Teams waste hours hunting for answers already in their files

->

THE SOLUTION

1. Recursively ingest any local folder — all supported file types
2. Chunk + embed documents using all-MiniLM-L6-v2 (local, free)
3. FAISS vector store enables millisecond semantic retrieval
4. GPT-4o-mini answers strictly from retrieved context — with citations

SYSTEM ARCHITECTURE



WHY RAG? — THE TECHNOLOGY THAT MATTERS

What Makes RAG Powerful

Retrieval before generation

The LLM never guesses — it only answers from retrieved document chunks, eliminating hallucinations.

No retraining required

Add new documents anytime. FAISS re-indexes on demand — no fine-tuning, no model updates.

Source citations built-in

Every answer references source_file, page number, chunk_id, and a text snippet for verification.

Local & cost-efficient

Embeddings use HuggingFace all-MiniLM-L6-v2 locally. Only LLM inference touches the OpenAI API.

RAG vs Plain LLM

Dimension	RAG	Plain LLM
Hallucinations	None	Common
Uses your data	Always	Never
Cites sources	Yes	No
Stays current	On rebuild	Cutoff
Cost	Low	High

TECHNICAL STACK

01 FRONTEND

Streamlit	Web UI — folder input, build button, Q&A chat

02 ORCHESTRATION

LangChain Classic	Ingestion, chunking, retriever chain
langchain-community	Loaders for DOCX, CSV, unstructured
langchain-text-splitters	Chunk size & overlap management

03 AI / EMBEDDINGS

HuggingFace all-MiniLM-L6-v2	Local sentence embeddings — zero API cost
OpenAI GPT-4o-mini	LLM for grounded answer generation

04 STORAGE & PARSING

FAISS-CPU	Local vector index — fast top-k semantic search

REAL-WORLD USE CASES

LG Legal & Compliance

Query contracts, NDAs, and regulations. Instantly surface relevant clauses with source page.

HR HR & Onboarding

Employees ask policy questions — the system retrieves exact handbook sections automatically.

FN Finance & Audit

Analysts interrogate reports and spreadsheets: "What was EBITDA in Q3?" — cited answer.

RS Research & Academia

Query across hundreds of papers or notes. Get summaries grounded in specific documents.

CS Customer Support

Support agents query product manuals and FAQs — answers backed by exact source snippets.

IT IT & DevOps

Query runbooks, incident reports, and architecture docs without hunting through folders.

IMPACT & CLOSING

Key Outcomes

0 — **hallucinations**
Answers grounded strictly in retrieved context

5+ — **file formats**
Unified query across all document types

Local — **embeddings**
No API cost for vector generation — HuggingFace

Full — **citations**
source_file, page, chunk_id with every answer

What This Project Demonstrates

1. End-to-end RAG pipeline design and implementation
2. Multi-format document ingestion and preprocessing
3. Vector store architecture (FAISS) and semantic search
4. LLM orchestration with LangChain for production use
5. Prompt engineering for strictly grounded responses
6. Full-stack deployment with Streamlit UI

github.com/Sajalagarwal-ca/RAG-Document-reader